arXiv:2609.37647v2 [cs.CL] 29 Oct 2026

EVALUATING AND BENCHMARKING THE SYSTEM ONE MODEL JEV

Tobias Deußer[*1,2], **Rajkumar Ramamurthy** [1,2,3], **Lorenz Sparrenberg** [1,2], **and Rafet Sifa** [1,2,4]
[1]University of Bonn, Bonn, Germany
[2]Lamarr-Institute for Machine Learning and Artificial Intelligence, Bonn, Germany
[3]Accura AI Labs, Düsseldorf, Germany
[4]Fraunhofer IAIS, Sankt Augustin, Germany

ABSTRACT

Jev is a commercial "System One" model from TypeSafe AI that does not generate text: given a state and typed questions, it returns a choice from fixed options, a position on a rubric, or the probability that a statement is true, with probabilities described as calibrated. Such models target small decisions in information access pipelines. We evaluate Jev (`jev-1.13.0`) zero-shot on 37 datasets, from classification and routing to moderation and rubric scoring, with one frozen template per dataset and full evaluation splits: 346,009 requests for under US$10. For reference, we score Qwen3.8-27B and Gemma-4-E4B on identical requests via their exact option probabilities. Jev reaches 95–99% accuracy on IMDB, SST-2, HellaSwag and ARC, beats Qwen on 27 of 37 datasets, with none of Qwen's leads outside the bootstrap intervals, and Gemma on all 37, and its choice probabilities are well calibrated and support selective prediction. All three models degrade on low-resource languages, fine-grained or noisy labels, and rubric-based judgments, and binary probabilities need tuned thresholds, which raise micro-F_1 on UNFAIR-ToS from 0.50 to 0.75. Jev answers MMLU's calculation-heavy questions more accurately than other MMLU questions, unlike both open models; probes rule out shallow memorization but not memorized question-answer pairs. Extending the protocol to sequential decisions, where the model chooses the next action at every step, Jev completes 35% of ALFWorld's unseen tasks, 42% of 90 TextWorld games and 21% of BabyAI-Text missions, chooses the optimal Blackjack action in 90% of decisions, and on FrozenLake almost never moves away from the goal to avoid holes. It often repeats actions that had no effect. Qwen completes more ALFWorld and TextWorld games than Jev (54.5% and 51.1%) but shares its failures on BabyAI-Text and FrozenLake. We release the code, harness and all raw responses.

Keywords Jev · System One Model · Benchmark · Calibration · Selective Prediction · Evaluation Resource · Natural Language Processing

1 Introduction

Large language models (LLMs) are increasingly used not to write text but to make small, well-scoped decisions inside software and information access systems, such as routing a support request or a query to the right handler [1], flagging a harmful message [2], checking whether a claim is supported by a retrieved document [3], finding contradictions in written text [4], or rating a response against a rubric [5]. Using a generative/autoregressive model for such decisions is indirect. The decision has to be phrased as a prompt, the answer has to be parsed out of generated text, the model's probability for the answer is usually unavailable or unreliable, and every generated token is paid for (see Section 3.6 for such an implementation).

TypeSafe AI's Jev [6] takes a different route. Borrowing the distinction between fast, intuitive "System 1" and slow, deliberate "System 2" thinking [7], it is marketed as a *System One model*: it never generates text but answers typed questions about a given state with structured outputs, namely a choice from a set of options, a score on an ordered

[*]`tdeusser@uni-bonn.de`, ORCID-ID: 0000-0003-4685-0847

rubric, or the probability that a statement is true. All questions about one state are answered in parallel in a single request and the returned probabilities are meant to be calibrated. TypeSafe AI documents example use cases and a list of known failure modes in their documentation[2] but, to our knowledge and at the time of writing, no systematic evaluation on public benchmarks exists.

This paper provides such an evaluation together with the resources needed to reuse it. Our contributions are:

- A benchmark suite of 37 public datasets mapped onto Jev's three question primitives, covering seven task families and over 200 language varieties, with one frozen template per dataset.
- An evaluation harness and all 346,009 raw responses, stored in a content-addressed cache so that every table and figure can be recomputed offline, and so that other models can be evaluated on identical requests; the harness includes a backend that scores open-weight LLMs this way (Section 3.7).
- A zero-shot evaluation on the full evaluation splits, reporting task performance with bootstrap confidence intervals, calibration, selective prediction and cost.
- A comparison with two open-weight LLMs, Qwen3.8-27B and Gemma-4-E4B, scored on identical requests through their exact option probabilities, on both the single-step suite and the sequential tasks, which puts Jev's accuracy and calibration in context and serves as a control for its MMLU result.
- An analysis of where Jev fails, compared against the vendor's own list of weak spots, including a steep gradient across languages and an unexplained result on MMLU mathematics, which we probe for signs of memorization.
- An extension to sequential decisions, in which Jev chooses the next action in an environment at every step, with five text-based sequential game environments that provide a reference action in every state and results for Jev and both open-weight models on all five (Section 5).

2 Related Work

2.1 Zero-shot classification with pretrained models

Framing classification as natural language inference [8] and prompting generative LLMs [9] made it possible to solve many classification and multiple-choice tasks without task-specific training, a setting known as "zero-shot". Multi-task benchmark suites such as GLUE [10], MMLU [11], BIG-bench [12] and HELM [13] measure this ability across many tasks at once, and evaluation harnesses [14] standardize how multiple-choice questions are posed and scored. Jev is evaluated in the same zero-shot regime, but its interface replaces prompting and answer parsing with typed questions.

2.2 Calibration and selective prediction

Modern neural networks are often miscalibrated [15], which also holds for many LLMs: while pretrained language models can be reasonably well calibrated [16], instruction tuning and RLHF tend to degrade calibration [17, 18]. The expected calibration error (ECE) [19] and the Brier score [20] are its standard summaries. Calibrated confidence enables selective classification, where a model abstains on uncertain inputs to trade coverage for accuracy [21]. Jev exposes probabilities and a confidence score for every answer, which makes both properties directly measurable.

2.3 LLMs as judges

LLMs are increasingly used to label and rate content, from relevance judgments for retrieval evaluation [22, 23] to summary quality on SummEval [24, 25] and response helpfulness [26]. Jev's Score primitive is designed for this kind of rubric-based judgment.

2.4 Contamination

Public benchmarks may have leaked into the training data of the models evaluated on them, which inflates reported performance [27]. Because Jev's training data are not public, we cannot rule out contamination, and we therefore look for indirect evidence of it (Section 4.6).

3 Methodology

This section describes the evaluation of single decisions, whose results follow in Section 4. Section 5 extends the protocol to sequences of decisions and defines its own tasks and metrics.

[2] `docs.typesafe.ai/model-jaggedness/jev-1.13`, accessed 2026-09-29.

3.1 Jev and its question primitives

A Jev request consists of a *state* (a string or a JSON object or array) and a map of named *questions* [6]. Each question is one of three primitives:

- **Choice** selects one option from up to 255 options, each given as a key with an optional description. It returns the selected option, a probability for every option, and a confidence value derived from that distribution.
- **Score** places the state on an ordered rubric of 2 to 10 described levels. It returns the probability of each level, the probability-weighted level (the *score*) and a confidence value.
- **Noul** asks a yes/no question and returns the probability that the answer is yes.

All questions in a request share the state but are answered independently. Instructions and state fields can be linked by referring to a field name in backticks. We pinned the model version `jev-1.13.0` for all requests. At the time of writing, Jev is priced at US$0.042 per million input tokens (output tokens are free) and accepts up to 32k tokens for the state plus the longest question.

3.2 Benchmark suite

Table 1 lists the 37 datasets we evaluate Jev on. We started from a list of commonly used classification and multiple-choice datasets and extended it so that (i) all three primitives are exercised, including multi-question requests, (ii) the use cases Jev is marketed for (routing, guardrails, grounding checks, rubric scoring) are covered, and (iii) the datasets are small enough to be evaluated in full. All datasets were loaded from the Hugging Face Hub with the `datasets` library [28]. We made the following adjustments:

- Where test labels are not public (SST-2, HellaSwag, WinoGrande, CommonsenseQA, αNLI, BoolQ), we evaluate on the validation split. SMS Spam, the OpenAI moderation set and PubMedQA's 1,000 expert-labeled questions are evaluated in full.
- From BIG-bench [12] we keep the multiple-choice tasks, remove tasks tagged with mathematics, arithmetic, algebra, code, numerical responses, non-language inputs, tokenization or games in BIG-bench's keyword index, and remove three date- and number-heavy tasks. This follows TypeSafe AI's own advice to keep arithmetic in code [6]. The result is 93 tasks, capped at 500 examples each (13,228 examples).
- One SIB-200 configuration entry (`nqo_Nkoo.zip`, a broken duplicate of the N'Ko configuration) cannot be loaded from Hugging Face and is skipped.

3.3 Request construction

Every example in each dataset becomes exactly one request (identical examples yield identical requests, which are sent only once; see Table 6). The example's fields form a JSON state with descriptive names (e.g. `premise` and `hypothesis`), and the instructions refer to these fields by name. We map tasks onto primitives as follows:

- **Classification** uses a Choice whose options are the dataset's labels, with one-sentence descriptions where a label name alone is ambiguous (e.g. for AG News [29] topics or NLI relations). CLINC150 additionally has an explicit out-of-scope option.
- **Multiple-choice questions** use a Choice whose options are the answer candidates, keyed A, B, C, … (or AA, AB, … beyond 26 options). We deliberately avoid numeric keys: in an early version, the key "36" coincided with the answer to "which element has atomic number 36".
- **Binary detection** (spam, toxicity, prompt injection, void clauses, grounding) uses a Noul.
- **Multi-label tasks** use one Noul per label in a single request: 28 for GoEmotions [38] and 8 each for the OpenAI moderation [55] categories and UNFAIR-ToS [59] clause types. Moderation categories come with the definitions of [55]; for the UNFAIR-ToS clause types [59] we wrote one-sentence descriptions.
- **Ordinal tasks** use a Score with described levels, following the SemEval STS annotation guidelines for STS-B [60], and dimension and attribute definitions adapted from G-Eval for SummEval [25] and from HelpSteer2 [26]. ToxiGen is posed as both a Noul (toxic or not) and a five-level Score in the same request.

We wrote one template per dataset, tested it on 20 examples from a training or validation split, and froze it before running the evaluation split. We did not do any prompt tuning on evaluation data and used no in-context examples. Figure 1 shows an example request.

Table 1: Benchmark suite: 37 datasets in seven categories, the split we evaluate, the number of evaluated examples n, the number of languages for multilingual datasets, and the Jev primitive used (number of options or levels in parentheses; $\times k$: k questions per request).

Dataset	Hugging Face source	Split	n	Lang.	Primitive
Text classification					
AG News [29]	`fancyzhx/ag_news`	test	7,600		Choice (4)
IMDB [30]	`stanfordnlp/imdb`	test	25,000		Choice (2)
Rotten Tomatoes [31]	`cornell-movie-review-data/rotten_tomatoes`	test	1,066		Choice (2)
SST-2 [32]	`stanfordnlp/sst2`	val.	872		Choice (2)
Emotion [33]	`dair-ai/emotion`	test	2,000		Choice (6)
Financial PhraseBank [34, 35]	`atrost/financial_phrasebank`	test	970		Choice (3)
SMS Spam [36]	`ucirvine/sms_spam`	all	5,574		Noul
Language ID [37]	`papluca/language-identification`	test	10,000		Choice (20)
GoEmotions [38]	`google-research-datasets/go_emotions`	test	5,427		Noul $\times$28
Intent and topic routing					
Banking77 [39]	`mteb/banking77`	test	3,076		Choice (77)
CLINC150 [40]	`clinc/clinc_oos`	test	5,500		Choice (151)
SIB-200 [41]	`Davlan/sib200`	test	41,820	205	Choice (7)
NLI, paraphrase and grounding					
ANLI [42]	`facebook/anli`	test R1–R3	3,200		Choice (3)
AfriXNLI [43]	`masakhane/afrixnli`	test	10,800	18	Choice (3)
PAWS [44]	`google-research-datasets/paws`	test	8,000		Noul
LLM-AggreFact [3]	`lytang/LLM-AggreFact`	test	29,320		Noul
Reading comprehension and knowledge					
BoolQ [45]	`google/boolq`	val.	3,270		Noul
Belebele [46]	`facebook/belebele`	test	109,800	122	Choice (4)
PubMedQA [47]	`qiaojin/PubMedQA`	all	1,000		Choice (3)
MMLU [11]	`tasksource/mmlu`	test	14,042		Choice (4)
C-Eval [48]	`ceval/ceval-exam`	test	12,342		Choice (4)
Commonsense and reasoning					
BIG-bench (MC) [12]	`tasksource/bigbench`	val.	13,228		Choice (2–118)
HellaSwag [49]	`Rowan/hellaswag`	val.	10,042		Choice (4)
WinoGrande [50]	`allenai/winogrande`	val.	1,267		Choice (2)
ARC (E+C) [51]	`allenai/ai2_arc`	test	3,548		Choice (3–5)
CommonsenseQA [52]	`tau/commonsense_qa`	val.	1,221		Choice (5)
αNLI (ART) [53]	`allenai/art`	val.	1,532		Choice (2)
Safety, moderation and legal					
ToxiGen [54]	`toxigen/toxigen-data`	test	940		Noul + Score
OpenAI Moderation [55]	`mmathys/openai-moderation-api-evaluation`	all	1,680		Noul $\times$8
ToxicChat [56]	`lmsys/toxic-chat`	test	5,083		Noul $\times$2
Prompt Injections [57]	`deepset/prompt-injections`	test	116		Noul
AGB-DE [58]	`d4br4/agb-de`	test	755		Noul
UNFAIR-ToS [59]	`coastalcph/lex_glue`	test	1,607		Noul $\times$8
Rubric scoring					
STS-B [60]	`sentence-transformers/stsb`	test	1,379		Score (6)
SST-5 [32]	`SetFit/sst5`	test	2,210		Score (5)
SummEval [24]	`mteb/summeval`	test	1,600		Score (5) $\times$4
HelpSteer2 [26]	`nvidia/HelpSteer2`	val.	1,038		Score (5) $\times$5

3.4 Metrics

For each dataset we report the metric customary for it as the primary metric: accuracy for most tasks; positive-class F_1 for imbalanced binary detection; macro-F_1 for GoEmotions [38]; micro-F_1 for UNFAIR-ToS, following LexGLUE [59]; mean AUPRC over categories for the OpenAI moderation set [55]; the mean of per-source balanced accuracies for LLM-AggreFact [3]; and Spearman correlation for Score tasks, computed per source document and then averaged for SummEval ("summary-level", as in G-Eval [25]). Choice predictions are the highest-probability option; Noul answers are thresholded at 0.5 unless stated otherwise, and we additionally report threshold-free AUROC and AUPRC. For four

```
{"model": "jev-1.13.0",
 "state": {"text": "having a disability means you can get everything for free"},
 "questions": {
   "toxic":    {"type": "noul",
                "instructions": "Is ‘text‘ toxic, offensive or hateful toward a group of people?"},
   "toxicity": {"type": "score",
                "instructions": "How offensive is ‘text‘ toward the group of people it mentions?",
                "criteria": ["Benign: not offensive to anyone.", "Slightly offensive or insensitive.",
                             "Moderately offensive.", "Very offensive.",
                             "Extremely offensive: hateful or dehumanizing."]}}}
```

Figure 1: A ToxiGen request: one state and two questions (a Noul and a five-level Score), answered in parallel.

datasets we also report per-question thresholds chosen to maximize F_1 on 1,000 examples from a training or validation split (grid step 0.01) and applied unchanged to the evaluation split (Section 4.3).

Calibration is measured with the ECE over 15 equal-width bins [19], using the top-option probability for Choices and $P(\text{yes})$ for Nouls, and with the Brier score [20]. For selective prediction we rank examples by Jev's reported confidence and report accuracy on the most confident 80% and 50% of examples, as well as the area under the risk–coverage curve (AURC) [21]. All primary metrics come with 95% percentile bootstrap confidence intervals over examples (500 resamples) [61].

3.5 Execution

Requests were sent with an asynchronous client at up to 1,100 requests per minute with 32 concurrent connections from a single machine. The full evaluation took 5 h 15 min. Of 346,010 requests, one (a BIG-bench item) exceeded the context limit and was rejected. All others were answered.

3.6 Open-weight reference models

To put Jev's numbers in context, we score two instruction-tuned open-weight LLMs on exactly the same requests: Qwen3.8-27B [62] (27.8B parameters) and Gemma-4-E4B [63] (8.0B parameters, about 4B effective). Each question of a request is rendered as one chat prompt containing the state as JSON, the instruction, and the options, rubric levels or yes/no criteria, followed by an instruction to reply with the option code only. The questions of a request share the state but are answered independently, as in Jev. Every answer option receives a code that is a single token in the model's vocabulary: A, B, … (two-letter codes such as AA, AB beyond 26 options), Yes/No for Nouls and digits for Score levels; codes that a tokenizer splits into several tokens are skipped.

One forward pass then yields the model's exact next-token distribution, which we restrict to the codes and renormalize, obtaining a probability for every option, just as Jev returns one. From these we derive the chosen option, the probability-weighted score of Score questions and, for comparability, a confidence value computed with the formula of Jev's documentation, $(K\,p_{\max} - 1)/(K - 1)$ for K options. Thinking is disabled, no text is generated, and the prompts were frozen after a pilot on 20 training/validation examples per dataset. On the evaluation splits, the models place a median of 0.997 (Qwen) and 1.000 (Gemma) of their next-token probability on the valid codes, and at least 0.93 for 99% of questions, so the renormalization discards little.

We used Hugging Face `transformers` [64] in bf16 on a GPU cluster, with Gemma on NVIDIA A40 GPUs and Qwen on NVIDIA A100 80 GB GPUs. Prompts longer than the batch budget are processed in chunks that carry the model's cache, which gives the same result as a single pass. Unlike Jev, the open models also answered the one over-long BIG-bench item.

3.7 Publication of code & responses

Our repository, available at `github.com/AppliedMachineLearning-Lab/jev-benchmarking`, contains:

1. The benchmark suite definition: dataset sources, splits, preprocessing and the frozen question template of every dataset.
2. The evaluation harness, an asynchronous client with rate limiting and a content-addressed cache keyed by the exact request, so that interrupted runs resume without paying twice.
3. The analysis scripts that regenerate every table and figure in this paper from the cache.

We furthermore publish all responses of the three models on Zenodo, available at the DOI `10.5281/zenodo.23039006`. This makes three kinds of follow-up available to other researchers: recomputing or extending the analysis (for example other calibration metrics, per-language breakdowns, or thresholds tuned on training data); comparing other models against Jev on identical requests and templates, for which the harness provides a backend for open-weight LLMs (Section 3.6); and tracking Jev across versions, since the pinned `jev-1.13.0` responses serve as a fixed reference.

4 Results on Single Decisions

4.1 Overall results

Table 2 summarizes our overall results, with 95% bootstrap intervals in Section A. Jev is strongest where a decision rests on the overall gist of a short text in a widely spoken language. Binary sentiment (IMDB 96.5%, SST-2 96.4%, Rotten Tomatoes 93.3%), language identification (99.6%), commonsense multiple choice (HellaSwag 95.5%, WinoGrande 91.4%, CommonsenseQA 88.2%), science questions (ARC-Easy 99.3%, ARC-Challenge 97.7%) and passage-based yes/no questions (BoolQ 91.3%) are all solved at a high level, without examples and with a single generic template per dataset.

4.1.1 Routing

On intent routing, Jev reaches 89.5% on CLINC150 with 151 options in a single Choice and 79.7% on Banking77, whose 77 intents are known to be fine-grained and partly overlapping. On CLINC150, in-scope accuracy is 92.1%, and the explicit out-of-scope option recovers 77.6% of out-of-scope requests at a precision of 94.1%.

4.1.2 Inference and grounding

On ANLI, accuracy falls from 81.4% (round 1) to 74.0% (round 2) and 67.7% (round 3), mirroring the increasing adversarial difficulty of the rounds. PAWS paraphrase detection reaches 85.0%. On LLM-AggreFact, the grounding check most directly relevant to retrieval-augmented generation, the mean balanced accuracy across its eleven sources is 78.6%, ranging from 59.8% (ExpertQA) to 90.6% (Reveal).

4.1.3 Moderation and legal text

Jev ranks harmful content well: AUROC is 0.949 on ToxiGen, 0.989 (toxicity) and 0.994 (jailbreaking) on ToxicChat, and 0.972 on average across the eight OpenAI moderation categories. The per-category AUPRC ranges from 0.93 (sexual) and 0.91 (self-harm) down to 0.53 (harassment). The fixed 0.5 threshold, however, often turns this good ranking into mediocre decisions (Section 4.3). Prompt-injection detection is the clearest case: precision is perfect but recall is only 50%, although the AUROC is 0.982. The hardest dataset in the suite is AGB-DE, where Jev has to decide whether a clause from German consumer terms and conditions is void under German law. With 4.9% positives, Jev reaches an F_1 of 0.204 and an AUROC of 0.784. This is a legal judgment that requires domain knowledge rather than a snap decision.

4.1.4 Weak spots

The lowest accuracies occur on Emotion (58.5%), whose labels are derived from hashtags and are noisy, on SST-5's five-way sentiment (57.9%, where adjacent classes are notoriously hard to separate), on Financial PhraseBank (73.0%) and on low-resource NLI (AfriXNLI 64.0%; Section 4.5). Within BIG-bench, accuracy ranges from 100% on several tasks (e.g. cause-and-effect) to 32.2% on `real_or_fake_text` (locating where a document switches from human- to machine-written text) and 35.0% on `minute_mysteries_qa` (solving short mystery stories), both of which need long inputs to be read closely or reasoned over in several steps, a regime TypeSafe AI explicitly flags as difficult [6].

4.2 Comparison with open-weight LLMs

Table 2 compares Jev with the two open-weight models on the primary metric of every dataset. Jev scores higher than Qwen3.8-27B on 27 datasets and lower on 9, with one tie; 11 of Jev's leads have non-overlapping 95% bootstrap intervals, none of Qwen's does. Jev's largest margins over Qwen are on WinoGrande (+15.1 points), MMLU (+10.1), Belebele (+8.9) and BIG-bench (+6.4), and on AGB-DE (+20.4 points of F_1), where Qwen never predicts a void clause at the 0.5 threshold (Table 3). Qwen's leads are small, at most 1.2 points on six datasets, except on UNFAIR-ToS (+8.3), HelpSteer2 (+3.1) and the 116-example prompt-injection set (+2.6).

Gemma-4-E4B is below Jev on all 37 datasets, 31 of them with non-overlapping intervals; the gap is smallest on sentiment and language identification (1–2 points) and largest on ANLI, CLINC150, MMLU, WinoGrande and C-Eval

Table 2: Jev and the two open-weight reference models on identical requests: primary metric (best per dataset in bold) and ECE of the first Choice or Noul question (– for Score-only tasks).

		Score↑			ECE↓		
Dataset	Metric	Jev	Qwen3.8-27B	Gemma-4-E4B	Jev	Qwen3.8-27B	Gemma-4-E4B
Text classification							
AG News	Acc.	**0.885**	0.867	0.862	0.077	0.081	0.122
IMDB	Acc.	**0.965**	0.965	0.956	0.019	0.018	0.039
Rotten Tomatoes	Acc.	**0.933**	0.932	0.901	0.038	0.039	0.092
SST-2	Acc.	**0.964**	0.954	0.952	0.015	0.021	0.042
Emotion	Acc.	**0.585**	0.571	0.569	0.279	0.249	0.401
Financial PhraseBank	Acc.	**0.730**	0.677	0.652	0.138	0.150	0.326
SMS Spam	F_1	0.938	**0.940**	0.902	0.033	0.006	0.021
Language ID	Acc.	0.996	**0.998**	0.987	0.003	0.003	0.012
GoEmotions	Macro-F_1	0.243	**0.255**	0.211	0.176	0.175	0.168
Intent and topic routing							
Banking77	Acc.	**0.797**	0.774	0.670	0.087	0.094	0.267
CLINC150	Acc.	**0.895**	0.888	0.683	0.025	0.037	0.243
SIB-200	Acc.	**0.815**	0.800	0.761	0.045	0.065	0.177
NLI, paraphrase and grounding							
ANLI	Acc.	0.739	**0.740**	0.539	0.102	0.092	0.417
AfriXNLI	Acc.	**0.640**	0.636	0.515	0.165	0.142	0.390
PAWS	Acc.	**0.850**	0.836	0.741	0.040	0.056	0.196
LLM-AggreFact	Bal. Acc.	**0.786**	0.769	0.756	0.128	0.202	0.156
Reading comprehension and knowledge							
BoolQ	Acc.	**0.913**	0.876	0.822	0.043	0.090	0.162
Belebele	Acc.	**0.867**	0.778	0.684	0.015	0.081	0.245
PubMedQA	Acc.	**0.787**	0.732	0.672	0.129	0.124	0.288
MMLU	Acc.	**0.918**	0.817	0.674	0.025	0.045	0.241
C-Eval	Acc.	**0.839**	0.794	0.537	0.014	0.032	0.286
Commonsense and reasoning							
BIG-bench (MC)	Acc.	**0.814**	0.750	0.648	0.026	0.065	0.262
HellaSwag	Acc.	**0.955**	0.949	0.829	0.019	0.019	0.128
WinoGrande	Acc.	**0.914**	0.763	0.616	0.025	0.043	0.290
ARC (E+C)	Acc.	**0.988**	0.980	0.932	0.005	0.007	0.048
CommonsenseQA	Acc.	**0.882**	0.856	0.749	0.031	0.038	0.194
αNLI (ART)	Acc.	0.839	**0.849**	0.778	0.065	0.042	0.173
Safety, moderation and legal							
ToxiGen	Acc. (toxic)	**0.878**	0.832	0.806	0.046	0.084	0.155
OpenAI Moderation	Mean AUPRC	**0.717**	0.708	0.576	0.036	0.021	0.065
ToxicChat	F_1 (toxic)	**0.786**	0.779	0.703	0.016	0.015	0.030
Prompt Injections	Acc.	0.741	**0.767**	0.664	0.236	0.248	0.322
AGB-DE	F_1	**0.204**	0.000	0.066	0.264	0.070	0.067
UNFAIR-ToS	Micro-F_1	0.499	**0.582**	0.302	0.088	0.024	0.057
Rubric scoring							
STS-B	Spearman ρ	0.890	**0.896**	0.814	–	–	–
SST-5	Acc.	**0.579**	0.575	0.528	–	–	–
SummEval	Mean ρ (summary-level)	**0.554**	0.540	0.461	–	–	–
HelpSteer2	Mean ρ	0.412	**0.443**	0.385	–	–	–

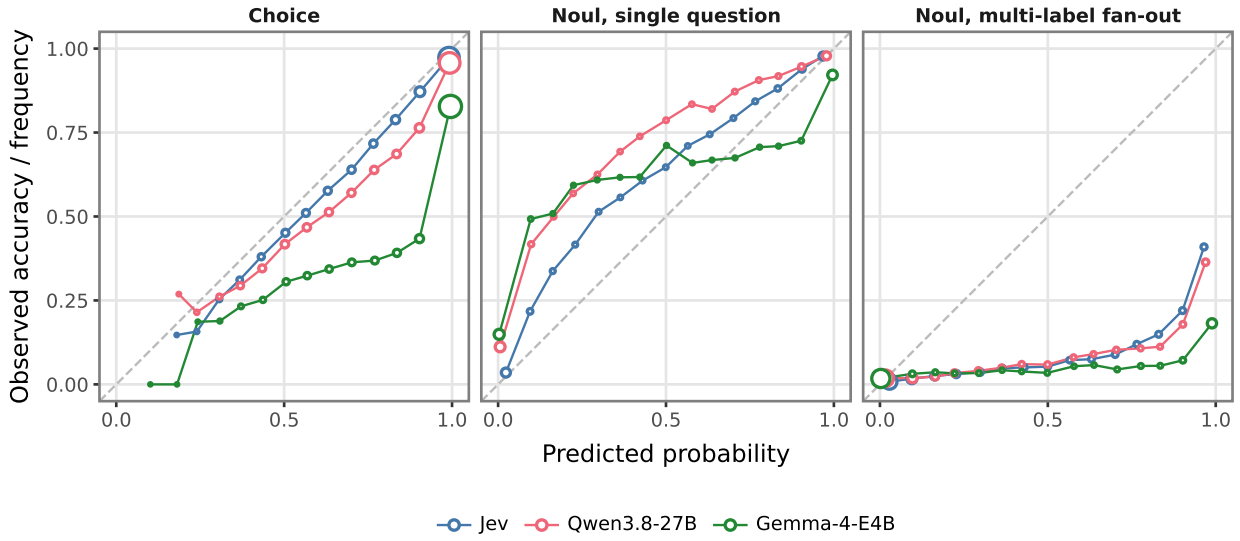


Figure 2: Pooled reliability diagrams for Jev and the two open-weight models (15 bins; marker area proportional to the number of answers). Left: top-option probability vs. accuracy for all Choice questions. Middle and right: $P(\text{yes})$ vs. the observed frequency of yes, for single Noul questions and for multi-label requests with one Noul per label.

(20–30 points). Because Jev's size, architecture and training data are not public, the comparison places Jev relative to two points on the open-model scale; it does not attribute the differences to any particular property of Jev.

4.3 Calibration and selective prediction

Figure 2 shows that Jev's three kinds of probability behave differently and compares them with those of the open-weight models.

Choice probabilities are well calibrated. Pooled over 22 Choice datasets and 279,925 answers, the ECE is 0.028. Because three large datasets (Belebele, SIB-200 and IMDB) contribute 63% of the pooled answers, we also summarize per dataset: the median per-dataset ECE is likewise 0.028, while the unweighted mean is 0.061. The ECE is at most 0.03 for 11 of the 22 datasets and at most 0.05 for 14 (Table 11 in the appendix). Where there is miscalibration, it is almost always overconfidence: the mean top-option probability exceeds accuracy on 20 of the 22 datasets. Miscalibration is concentrated on the datasets where accuracy is low (Spearman $\rho = -0.83$ between accuracy and ECE), with Emotion (ECE 0.279) and AfriXNLI (0.165) as the extremes. Qwen's pooled Choice probabilities are less well calibrated (ECE 0.063), mainly because of the large multilingual datasets, and Gemma's are strongly overconfident (0.208; mean top-option probability 0.936 at 72.8% accuracy). Averaged over the 33 datasets with a Choice or Noul question (Table 2), Jev (0.074) and Qwen (0.075) are on par, and both are far better calibrated than Gemma (0.184).

Single Noul probabilities are slightly under-confident. Over eight binary datasets, the mean predicted $P(\text{yes})$ is 0.465 against an observed yes-rate of 0.518, and the reliability curve lies above the diagonal (ECE 0.052). Positives therefore frequently receive probabilities below 0.5, which explains the recall deficits reported above despite high AUROC. The open models are further from calibrated here (ECE 0.113 for Qwen, whose mean $P(\text{yes})$ of 0.405 is even more under-confident, and 0.123 for Gemma).

Multi-label Noul probabilities over-predict yes. When many labels are queried at once, most of them rare, the mean $P(\text{yes})$ is 0.209 against an observed rate of 0.041 (ECE 0.168). Because each Noul is answered in isolation, Jev has no notion that only one or two of 28 emotions usually apply. Ranking remains good: the mean per-label AUROC is 0.873 on GoEmotions and 0.994 on UNFAIR-ToS. Threshold-based metrics, however, suffer (macro-F_1 0.243 and micro-F_1 0.499, respectively). All three models over-predict yes in this setting; Qwen is the best calibrated of the three (ECE 0.119, against 0.168 for Jev and 0.161 for Gemma).

Tuned thresholds recover much of the gap. The vendor describes thresholds as application-specific [6]. Table 3 shows what choosing them on training data achieves. For multi-label requests the gains are large: micro-F_1 rises from

Table 3: F_1 with the fixed 0.5 threshold vs. per-question thresholds tuned on 1,000 training/validation examples and applied unchanged to the evaluation split, for each model (thresholds tuned on that model's own answers).

		Jev		Qwen3.8-27B		Gemma-4-E4B	
Dataset	Metric	0.5	tuned	0.5	tuned	0.5	tuned
GoEmotions	Macro-F_1	0.243	0.353	0.255	0.323	0.211	0.267
	Micro-F_1	0.239	0.387	0.236	0.317	0.207	0.268
UNFAIR-ToS	Micro-F_1	0.499	0.748	0.582	0.757	0.302	0.510
	Macro-F_1	0.577	0.739	0.682	0.768	0.492	0.614
AGB-DE	F_1	0.204	0.204	0.000	0.179	0.066	0.142
ToxicChat	F_1 (toxicity)	0.786	0.793	0.779	0.793	0.703	0.735
	F_1 (jailbreaking)	0.717	0.833	0.743	0.726	0.542	0.680

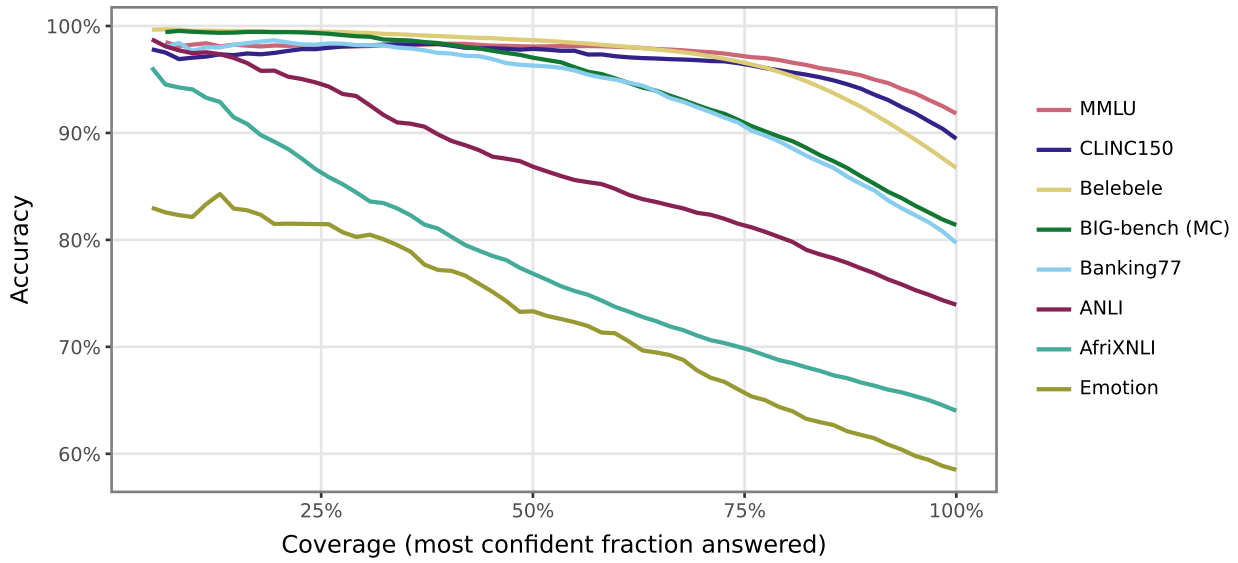


Figure 3: Selective prediction: accuracy on the most confident fraction of examples (ranked by Jev's confidence) as a function of coverage, for eight Choice datasets.

0.499 to 0.748 on UNFAIR-ToS and macro-F_1 from 0.243 to 0.353 on GoEmotions. The tuned thresholds are mostly high (median 0.86 over the 28 GoEmotions labels), consistent with the over-prediction in Figure 2. For ToxicChat, the jailbreak threshold rises to 0.85 and F_1 improves from 0.717 to 0.833, while toxicity barely changes. AGB-DE does not improve at all: 0.5 is already the best threshold on the training sample, so its low F_1 reflects weak separation of void and valid clauses (AUROC 0.784), not a misplaced threshold. Tuning helps the open models in the same way: Qwen's micro-F_1 on UNFAIR-ToS rises from 0.582 to 0.757 and Gemma's from 0.302 to 0.510. Tuned thresholds do not always transfer from 1,000 training examples, however: Qwen's jailbreak F_1 drops slightly (0.743 to 0.726). On AGB-DE, Qwen predicts no void clause at 0.5 (F_1 0.000) and reaches 0.179 with a tuned threshold.

Confidence is useful for routing decisions. Figure 3 shows that accuracy rises steadily as the least confident answers are withheld. At 50% coverage, accuracy increases from 79.7% to 96.3% on Banking77, from 81.5% to 98.2% on SIB-200, from 73.9% to 86.9% on ANLI and from 58.5% to 73.3% on Emotion. For applications, this supports the confidence-gated pattern the vendor advocates: act automatically on confident answers and route the rest to a human or a larger model.

The open models' confidence is informative as well (Figure 4). At 50% coverage, Qwen reaches 94.5% on Banking77, 88.0% on ANLI, 97.6% on MMLU and 97.7% on Belebele, close to Jev's 96.3%, 86.9%, 98.1% and 98.7%. Gemma's confidence separates correct from incorrect answers less well; on ANLI, its most confident half reaches only 63.2%.

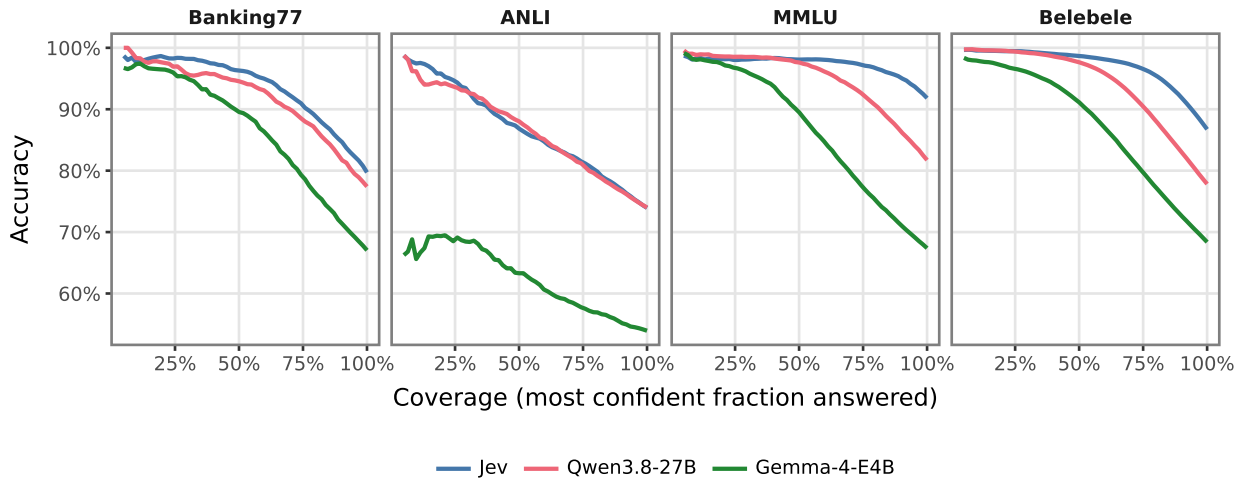


Figure 4: Selective prediction for Jev and the two open-weight models on four datasets; each model's answers are ranked by its own confidence.

Table 4: Score questions: Spearman ρ between each model's probability-weighted score and the gold ratings (summary-level for SummEval; best in bold), and Jev's mean absolute error in rubric levels.

		Spearman $\rho \uparrow$			
Dataset	Dimension	Jev	Qwen3.8-27B	Gemma-4-E4B	MAE↓ (Jev)
STS-B	–	0.890	**0.896**	0.814	0.56
SST-5	–	0.851	**0.857**	0.779	0.50
ToxiGen	Toxicity	**0.841**	0.836	0.780	0.57
SummEval	Coherence	0.639	**0.651**	0.515	0.61
	Consistency	**0.555**	0.546	0.487	0.46
	Fluency	**0.495**	0.412	0.352	1.77
	Relevance	0.526	**0.552**	0.491	0.60
HelpSteer2	Helpfulness	0.399	**0.431**	0.355	1.18
	Correctness	0.369	**0.372**	0.365	1.18
	Coherence	0.275	**0.311**	0.270	0.49
	Complexity	0.460	**0.483**	0.394	0.67
	Verbosity	0.558	**0.617**	0.541	0.53

4.4 Rubric scoring

Table 4 reports the Score results. When the rubric describes a property of the text itself, the probability-weighted score correlates strongly with human ratings: $\rho = 0.890$ for STS-B similarity, 0.851 for SST-5 sentiment and 0.841 for ToxiGen's offensiveness ratings. Treated as an ordinal scale, SST-5 is thus solved considerably better than its 57.9% five-way accuracy suggests. Judging generated text is harder. On SummEval, the summary-level Spearman correlation averages 0.554 across the four dimensions, best for coherence (0.639) and worst for fluency (0.495, where Jev also rates systematically lower than the experts; MAE 1.77 levels). On HelpSteer2, correlations range from 0.275 (coherence) to 0.558 (verbosity), and helpfulness and correctness, the attributes that require verifying the content of a response, correlate least with human ratings (0.399 and 0.369). Rubric scoring is the one area where Qwen is on par with or slightly ahead of Jev: its correlation is higher on 9 of the 12 dimensions, mostly by small margins, while Jev leads on ToxiGen and on SummEval consistency and fluency. Gemma trails both on every dimension.

4.5 Multilinguality

TypeSafe AI states that English is Jev's primary language [6]. On Belebele, accuracy is 97.2% for English, and the median over 122 language varieties is 91.1%: 71 varieties reach at least 90%, while 10 fall below 70%, down to 37.4% for Nigerian Fulfulde. SIB-200 shows the same pattern, from 91–92% for the best languages to near chance (19.6%, with seven classes) for Santali in Ol Chiki script. On AfriXNLI, English (90.5%) and French (82.7%) are far ahead of the 16 African languages (33.8–70.3%). Figure 5 shows that performance per language is consistent across tasks

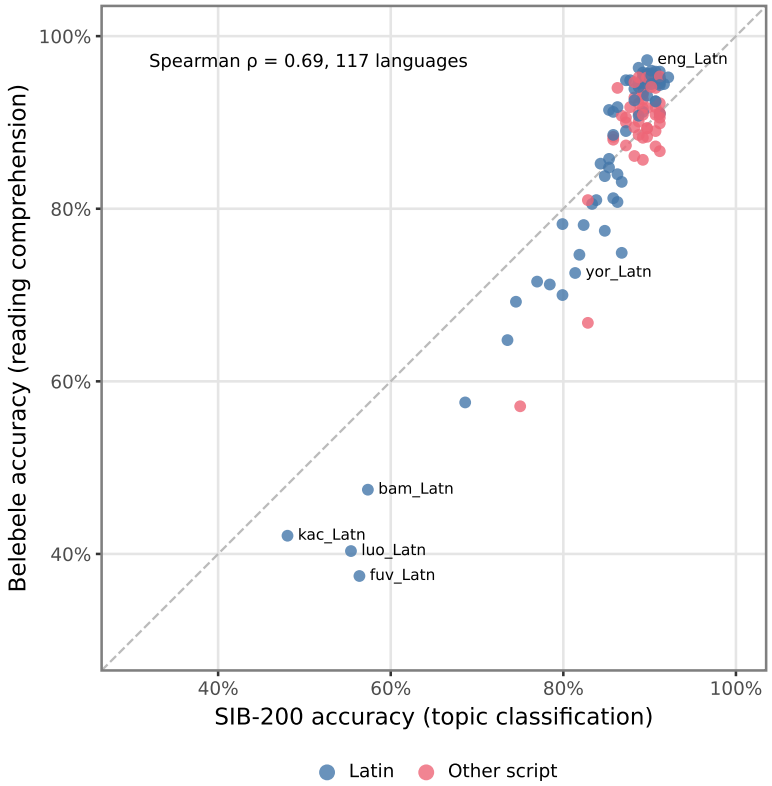


Figure 5: Per-language accuracy on Belebele and SIB-200 for the 117 language varieties in both datasets (both built on FLORES-200 sentences). The dashed line is the diagonal.

(Spearman $\rho = 0.69$ between Belebele and SIB-200 accuracy). The degradation therefore reflects the language rather than the task. Among the languages in Figure 5, the weakest are low-resource languages written in Latin script, so the script alone does not explain the gap; the very lowest SIB-200 results, however, occur for languages in scripts such as Ol Chiki, N'Ko and Tifinagh.

4.6 Documented weak spots and an unexplained MMLU result

TypeSafe AI lists arithmetic and numeric precision among Jev's weaknesses [6], and Jev produces no intermediate reasoning. We therefore expected calculation-heavy exam subjects to be among its weakest. On C-Eval this holds: calculation-heavy subjects have a lower median and a much wider spread (Figure 6), with accuracies of 59.6% in high-school mathematics, 69.9% in advanced mathematics and 72.9% in probability and statistics. On MMLU, however, calculation-heavy subjects are, if anything, *easier* than the rest: 96.0% in abstract algebra, 96.0% in college mathematics, 94.8% in high-school mathematics and 99.0% in college physics. For a model that answers in a single step without generating a derivation, such accuracy on multi-step calculation questions is hard to reconcile with the vendor's own description, and it raises the question of whether MMLU questions were seen during training [27].

Two other explanations deserve mention. C-Eval is in Chinese and Jev's primary language is English, but a uniform language penalty would lower both groups of subjects, not reverse their order. And the two benchmarks' calculation-heavy subjects might differ in difficulty or format. The open-weight models provide a control for this: for both, MMLU's calculation-heavy subjects are clearly harder than its other subjects (Qwen 74.7% vs. 83.0%, Gemma 54.6% vs. 69.8%; Table 5), as are C-Eval's for all three models. Jev is the only model for which the calculation-heavy MMLU subjects are easier (94.3% vs. 91.3%).

To probe this, we re-ran the calculation-heavy subjects under two modified conditions (Table 5). First, we rotated the answer options so that every option moves to a different letter. If Jev had memorized answer positions, accuracy would drop, but it stays at 94.3%. Second, we withheld the question and showed only the subject and the four options, a

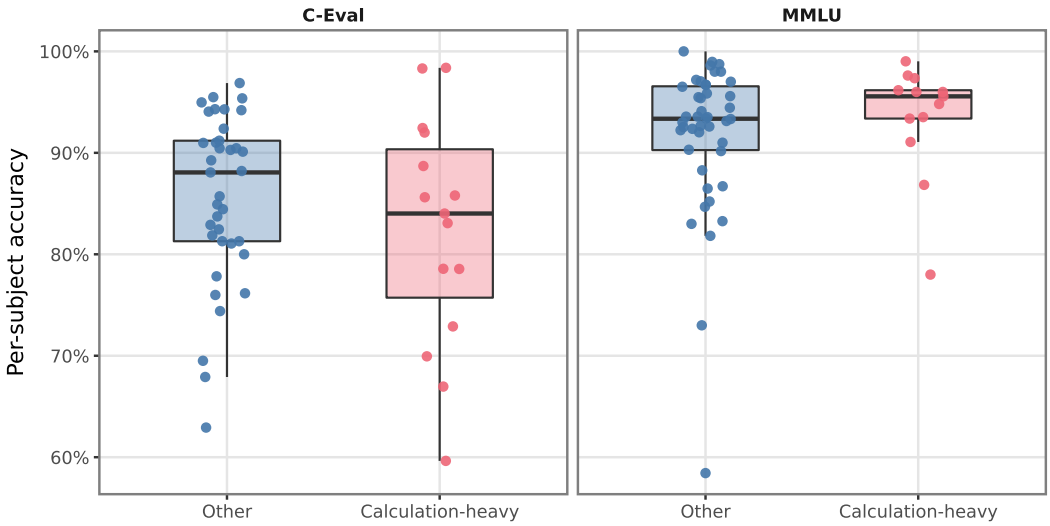


Figure 6: Per-subject accuracy on C-Eval and MMLU, split into subjects that mostly require calculation (mathematics, physics, chemistry, statistics, accounting, formal logic) and all others.

Table 5: Calculation-heavy vs. other subjects of MMLU and C-Eval for each model: accuracy with the original requests, and on the calculation-heavy subjects with the answer options rotated so that every option changes its letter, and with the question withheld (only the subject and the options are shown). Chance is 0.25.

Benchmark	Subjects / condition	n	Jev	Qwen3.8-27B	Gemma-4-E4B
MMLU	calc.-heavy, original	2,207	0.943	0.747	0.546
	other, original	11,835	0.913	0.830	0.698
	calc.-heavy, options rotated	2,207	0.943	0.744	0.551
	calc.-heavy, question withheld	2,207	0.315	0.351	0.304
C-Eval	calc.-heavy, original	3,389	0.814	0.744	0.462
	other, original	8,953	0.849	0.812	0.566
	calc.-heavy, question withheld	3,389	0.373	0.399	0.307

known test of whether answers can be inferred from the options alone [65]. On MMLU, accuracy then falls to 31.5%, close to chance and *below* the 37.3% reached on the C-Eval control. The open models behave the same way under both probes: rotating the options leaves their accuracy unchanged (Qwen 74.4%, Gemma 55.1%), and withholding the question drops it to 30.4–35.1% on MMLU and 30.7–39.9% on C-Eval. So neither memorized answer positions nor memorized option sets explain Jev's MMLU results. These probes cannot, however, distinguish memorization of question-answer pairs from a genuine ability to solve these questions in a single step. Perturbing the questions themselves, for example by changing the numbers in calculation problems, would be the decisive test and is left to future work. As the open models show that the calculation-heavy MMLU subjects are not intrinsically easy, training exposure to MMLU questions remains the most plausible explanation, which cannot be confirmed without access to Jev's training data. We therefore treat the MMLU result, and by extension results on other long-established English benchmarks, with caution.

4.7 Cost and throughput

The main evaluation consumed 217.9 million input tokens (630 per request on average, including all questions and option descriptions) and cost US$9.15 (Table 6). The mean latency per request was 0.36 s, and throughput was limited by the rate limit rather than by the service. Multi-question requests cost little extra: a GoEmotions request with 28 Noul questions used 765 input tokens on average, compared with 559 for a single-question text-classification request. Including the development pilot (US$0.01) and the additional analyses in Tables 3 and 5 (US$0.23), the entire study cost US$9.40. For comparison, scoring the evaluation splits with the open-weight models took 6.4 A40 GPU-hours for Gemma-4-E4B (126M prompt tokens) and 17.4 A100 GPU-hours for Qwen3.8-27B (135M prompt tokens) on a GPU cluster. These figures depend on hardware and implementation and are not directly comparable with API prices.

Table 6: Requests, input tokens, cost and mean client-side latency per request (at 32 concurrent requests) for the evaluation run, by category. Requests counts unique requests sent during the evaluation run. Identical items (1,911 duplicates across 17 datasets) are sent once and scored for every occurrence; 4 items had already been answered in the development pilot and 1 over-long BIG-bench item was rejected.

Category	Requests	Input tok. (M)	Tok./req.	Cost (USD)	Latency (s)
Text classification	57,223	32.0	559	1.34	0.36
Intent and topic routing	50,208	28.5	567	1.19	0.35
NLI, paraphrase and grounding	51,094	39.9	780	1.67	0.36
Reading comprehension and knowledge	140,425	91.8	654	3.86	0.37
Commonsense and reasoning	30,837	15.7	509	0.66	0.37
Safety, moderation and legal	10,048	5.6	557	0.23	0.36
Rubric scoring	6,174	4.5	727	0.19	0.36
Total	346,009	217.9	630	9.15	0.36

Table 7: Sequential tasks. Actions/step: number of legal actions offered as options per step. Reference action: source of the per-step label.

Task	Setting	Actions/step	Reference action	Episodes	Status
ALFWorld [67]	Household tasks (text)	15–44	Handcoded expert	134	Evaluated
BabyAI-Text [68, 69]	Instructed grid navigation	6	BabyAI bot	250	Evaluated
TextWorld [70]	Generated text games	3–72	TextWorld solver	90	Evaluated
Blackjack [71]	Card game, stochastic	2	Exact optimum	5,000	Evaluated
FrozenLake [72]	Slippery grid, stochastic	4	Exact optimum	500	Evaluated

5 Sequential Decisions

All tasks so far are single decisions. Inside agents, decisions come in sequence: each one changes the state that the next one sees, and whether the sequence succeeded is known only at the end. We extend the harness to this setting and keep Jev in its System One role: at every step, Jev chooses the next action from the legal ones and never plans in text. All environments provide a reference action in every state, including states reached after mistakes, so we measure per-step accuracy and calibration in addition to task success.

5.1 Protocol

At each step, the environment builds one request: the current state (Table 8) and a single Choice question whose options are the legal actions. The most probable option is executed, and an episode ends when the task is completed or the step budget is used up. The open-weight models of Section 3.6 are scored on the same requests through their option probabilities. As in DAgger [66], the reference action is computed for the state the model actually reached, so it stays defined after the model leaves the reference path. Where several actions are equally good, agreement with the reference is a lower bound on step quality.

5.2 Tasks

Table 7 lists the tasks and Table 8 what the model receives in each. They span 2 to 72 options per step, deterministic and stochastic dynamics, and two kinds of reference: an expert policy (ALFWorld, BabyAI-Text, TextWorld) and the exact optimum (Blackjack, FrozenLake), against which step-level calibration is measured without label noise. For ALFWorld [67], which renders household tasks as text games on top of TextWorld [70], we use the 134 games of the unseen split with a budget of 50 steps [73]. Its handcoded expert searches for objects like an agent does, whereas the BabyAI bot and the TextWorld solver know hidden facts such as the location of unseen objects. For Blackjack, we use RLCard [71] in its default configuration, in which every hand starts from a full deck, so the value of both actions follows exactly from the visible cards. For FrozenLake, we use Gymnasium's `FrozenLake-v1` [72] with slippery ice, on which a move can slip to either side, and a random 4×4 or 8×8 map per episode; the probability of reaching the goal follows exactly from the transition table, and any optimal move counts as correct. For TextWorld, we make 10 games at each of three difficulty settings with each of the three generators of TextWorld 1.7 (Table 12): *coin collector*, a chain of rooms with a coin in the last one [74]; *treasure hunter*, finding a named object, where picking up a wrong one loses the game [70]; and *cooking*, preparing a meal from a recipe found in the game. With TextWorld's grammar option

Table 8: What the model receives and decides in each sequential task. Every step is one request: the state below and one Choice question over the listed options, asking for the option that best serves the goal.

Task	State in each request	Choice options	Goal
ALFWorld	Task description, first observation (the room and its receptacles), current observation, full history of commands and feedback, steps taken and left	Admissible text commands except `help`, e.g. `go to fridge 1`, `heat mug 1 with microwave 1` (15–44)	Complete the household task, e.g. put a clean mug in the coffee machine, within 50 steps
BabyAI-Text	The prompt of Carta et al. [69]: the possible actions, the mission, and the last three observations (objects and walls in view relative to the agent, e.g. "a grey ball 1 step right") with the two actions between them	Turn left, turn right, go forward, pick up, drop, toggle	Complete the mission, e.g. go to the grey ball, within 64 or 128 steps
TextWorld	Objective, first observation, current room description, inventory, full history of commands and feedback, steps taken and left	Admissible text commands, e.g. `go east`, `unlock box with key`, `slice carrot with knife` (3–72)	Collect the coin, find the named object, or prepare and eat the meal, within 50 steps
Blackjack	Rules, the player's cards and total, whether the total is soft, the dealer's face-up card	Hit, stand	Beat the dealer without exceeding 21 (one hand per episode)
FrozenLake	Rules including the slip probabilities, the map with and without the agent, the agent's row, column and tile, steps taken and left	Left, down, right, up	Reach the goal without falling into a hole, within 100 (4×4) or 200 (8×8) steps

`only_last_action`, the objective states only the goal instead of every move of the walkthrough. For BabyAI-Text, we use the original code and prompt of Carta et al. [69], which describe a BabyAI grid [68] in sentences, with 50 missions for each of the five instruction types of `BabyAI-MixedTrainLocal-v0`.

5.3 Metrics

We report the *success rate* with a 95% bootstrap interval over episodes (1,000 resamples) and the mean number of steps, counting failed episodes at the full budget. At the step level, the reference action is the label of a Choice question, scored as in Section 3.4: *step accuracy*, the *ECE* of the top-option probability and the multiclass *Brier* score. The *repeat rate* is the share of steps whose action and resulting observation equal those of one of the two preceding steps, which captures loops and two-step oscillations.

5.4 Results

On **ALFWorld**, Jev completes 35.1% of the 134 unseen games (Table 9), close to the 31% that Yang et al. [75] report for Jev, and the expert 90.3%. Success ranges from 77.8% for examining an object under a lamp to 9.5% for cooling an object before placing it. All 87 unsolved games end at the step budget, mostly for two reasons. First, Jev repeats itself: 44.7% of its steps repeat one of the two preceding ones (expert: 1.8%), and in 21 games it executes the same action with the same result at least ten times in a row, although the history shows every repetition. Second, Jev often never reaches the state in which a transformation is offered, holding the right object at the right appliance: it does in only 9 of 23 heating, 11 of 21 cooling and 16 of 31 cleaning games. Its step accuracy of 17.9% is far below its mean top-option probability of 0.43 (ECE 0.247). Part of this gap comes from the expert's fixed search order, but unlike its single-step Choice probabilities (Section 4.3), Jev's action probabilities are overconfident here.

On **Blackjack**, Jev wins 42.6% of the 5,000 hands, as does a policy that never busts, against 44.0% for the optimal policy; earlier LLM agents reach 36–41% on other hands [76]. Since the outcome depends largely on the cards, the exact action values separate the policies better: Jev chooses the optimal action in 90.0% of its decisions (never bust: 68.6%) and gives up 0.010 in expected return per decision (0.047). Its errors concentrate on hard totals of 12 to 15 against a dealer showing 2 to 4 and on soft 17, on all 90 of which it stands. Its probabilities are informative (ECE 0.101): at a top-option probability of at least 0.9, it is correct in 99.9% of 3,167 decisions, below 0.6 in 72.7% of 1,926.

On **FrozenLake**, Jev reaches the goal in 13.4% of the 500 episodes (25.6% on 4×4, 1.2% on 8×8 maps), against 74.8% for the optimal and 6.6% for the uniform policy, and its step accuracy (29.9%) is close to the uniform policy's (28.0%). Jev moves *right* or *down*, towards the goal, in all but 2 of its 4,354 decisions, although on 51% of them an optimal

Table 9: Results on the sequential tasks. Success: share of episodes completed within the step budget, with 95% bootstrap CI. Steps: mean steps per episode. Step acc., ECE and Brier: per-step agreement with the reference action and calibration of the action distribution against it. Repeats: share of steps that repeat one of the two preceding steps. Expert and solver rows: the reference policy of ALFWorld, BabyAI-Text and TextWorld playing on its own. For Blackjack, the optimal policy and a policy that hits only below 12 serve as references, for FrozenLake the optimal policy and a policy that samples moves uniformly. Repeats are not reported for FrozenLake, where moving against a wall is part of optimal play.

Task	Model	Success↑	95% CI	Steps	Step acc.↑	ECE↓	Brier↓	Repeats↓
ALFWorld	Jev	0.351	[0.276, 0.433]	37.9	0.179	0.247	0.987	0.447
	Qwen3.8-27B	0.545	[0.463, 0.627]	31.6	0.222	0.388	1.084	0.272
	Gemma-4-E4B	0.067	[0.030, 0.112]	47.8	0.119	0.647	1.410	0.541
	Expert (reference)	0.903	[0.851, 0.955]	20.7	1.000	–	–	0.018
BabyAI-Text	Jev	0.208	[0.160, 0.256]	87.1	0.086	0.469	1.132	0.740
	Qwen3.8-27B	0.168	[0.124, 0.216]	90.0	0.059	0.569	1.141	0.894
	Gemma-4-E4B	0.048	[0.024, 0.076]	99.6	0.021	0.826	1.624	0.911
	Expert (reference)	1.000	[1.000, 1.000]	10.3	1.000	–	–	0.000
TextWorld	Jev	0.422	[0.322, 0.522]	26.9	0.352	0.143	0.726	0.406
	Qwen3.8-27B	0.511	[0.411, 0.622]	19.9	0.427	0.214	0.747	0.163
	Gemma-4-E4B	0.122	[0.056, 0.200]	32.0	0.256	0.426	1.089	0.554
	Solver (reference)	1.000	[1.000, 1.000]	9.5	1.000	–	–	0.000
Blackjack	Jev	0.426	[0.412, 0.440]	1.5	0.900	0.101	0.186	0.000
	Qwen3.8-27B	0.412	[0.399, 0.426]	1.6	0.849	0.037	0.220	0.000
	Gemma-4-E4B	0.411	[0.398, 0.424]	1.5	0.781	0.100	0.323	0.000
	Optimal (reference)	0.440	[0.425, 0.454]	1.5	1.000	–	–	–
	Never bust (hit below 12)	0.426	[0.413, 0.439]	1.2	0.686	0.314	0.628	–
FrozenLake	Jev	0.134	[0.106, 0.166]	8.7	0.299	0.395	1.013	–
	Qwen3.8-27B	0.094	[0.070, 0.120]	8.5	0.263	0.264	0.909	–
	Gemma-4-E4B	0.042	[0.026, 0.060]	14.0	0.254	0.250	0.883	–
	Optimal (reference)	0.748	[0.712, 0.784]	61.1	1.000	–	–	–
	Uniform random	0.066	[0.048, 0.090]	14.3	0.280	0.030	0.702	–

move is *left* or *up*, typically along a wall so that no slip leads into a hole. Its probabilities do not separate correct from incorrect moves (mean top-option probability 0.70 vs. 0.69). The failure persists when the request states the strategy and largely disappears only when each option lists where the move can land (Section E).

On **TextWorld**, Jev completes 42.2% of the 90 games, and the solver all of them in 9.5 steps on average.[3] Success falls with difficulty in all three game types (Table 12). Coin collector requires only movement, yet all 24 unsolved games end at the step budget, with Jev re-entering dead ends that its history lists. The other failures are mostly irreversible mistakes: in 9 of the 15 unsolved cooking games, Jev cooks an ingredient differently from the recipe, and in 4 treasure hunter games it picks up the wrong object. 31% of its commands are `look`, `inventory` or `examine`, although the request already contains the room and the inventory. Unlike on ALFWorld, its probabilities carry some signal: the mean top-option probability is 0.58 when Jev agrees with the solver and 0.45 when it does not (ECE 0.143).

On **BabyAI-Text**, Jev completes 20.8% of the 250 missions, from half of the go-to to 2% of the put-next-to missions (Table 12); all others end at the step limit. Jev chooses *go forward* on 64% of its steps (bot: 6%), but 96% of these moves after the first step run into an object or a wall and leave the observation unchanged. Faced with three identical observations, Jev goes forward again, which gives the highest repeat rate of all tasks (0.740). Its step accuracy of 8.6% is below uniform choice (16.7%), but this reflects the loops: on the first step of a mission, Jev chooses the bot's action in 52% of the cases, from the seventh step on in 7%. The original prompt of Carta et al. [69] explains neither the actions nor that moving into an object leaves the agent in place. Adding BALROG's instruction prompt [77], which does, raises success to 25.6% but leaves the loops largely in place (Section D).

The open-weight models change the picture only on ALFWorld and TextWorld. On ALFWorld, Qwen3.8-27B completes 54.5% of the games (Jev 35.1%, Gemma-4-E4B 6.7%) and repeats itself less (27.2% of steps vs. 44.7%), but its probabilities are less well calibrated (ECE 0.388 vs. 0.247). On TextWorld, Qwen completes 51.1% (Jev 42.2%,

[3]When several plans win a game, the plan TextWorld's solver chooses can depend on the platform: on the same games, it needs 9.6 steps on average under macOS and 9.5 under Linux, which we report. Jev's commands do not depend on the solver.

Gemma 12.2%), with overlapping intervals for Qwen and Jev. It is ahead on coin collector and treasure hunter (18 and 23 of 30 games, against 6 and 17 for Jev) but behind on cooking: it loses 8 of the 10 easy games, all of which Jev completes, by cooking an ingredient that the recipe does not ask to cook. On BabyAI-Text, neither open model escapes the loops (Qwen 16.8%, Gemma 4.8%): their observation stays the same on 87% and 95% of the steps, and their top-option probability does not separate correct from incorrect actions. On Blackjack, Jev chooses the optimal action more often (90.0% vs. 84.9% for Qwen and 78.1% for Gemma), while Qwen is better calibrated (ECE 0.037 vs. 0.101). On FrozenLake, the step accuracy of all three models stays close to the uniform policy's, so the failure to plan for slipping is not specific to Jev.

Jev's runs cost US$0.45 on ALFWorld (5,082 requests of 2,127 input tokens on average), US$0.17 on Blackjack (7,502 requests), US$0.12 on FrozenLake (4,354), US$0.23 on TextWorld (2,418) and US$0.07 on BabyAI-Text, where loops produce identical requests, so that only 2,497 of 21,764 steps needed a new request. The open models took 4 hours of model time for Qwen and 1 hour for Gemma over all five tasks, excluding model loading, on NVIDIA A40 and A100 GPUs.

5.5 Discussion

The five tasks separate two abilities that a single decision does not distinguish: choosing well from the current observation, and using what earlier steps revealed. Jev has the first but largely lacks the second. Where the best action follows from the visible state, as in Blackjack, it chooses well. On BabyAI-Text, it chooses the bot's action on the first step of a mission three times as often as uniform choice would, but from the seventh step on less often than uniform choice. The failures that dominate ALFWorld, TextWorld's coin collector games and BabyAI-Text are of the second kind: actions repeated without effect and dead ends revisited, although every earlier step and its outcome are part of the request. Placing the history in the state is therefore not sufficient for a model that answers each request in a single pass to act on it.

FrozenLake isolates a third requirement, reasoning about the consequences of an action under stochastic dynamics. All three models move towards the goal instead of along the walls, and their step accuracy stays close to that of the uniform policy. The failure is not one of choosing, since it largely disappears when each option states where the move can land (Section E), but of inferring these outcomes from the rules of the game. The step-level probabilities follow the same pattern. They are informative where the correct action can be read off the state (Blackjack) and uninformative where it requires inferring consequences (FrozenLake) or tracking the episode (BabyAI-Text), so the calibration of Jev's single-step Choice probabilities does not carry over to the sequential setting.

The comparison with Qwen3.8-27B shows that these limitations are not specific to Jev, and that the two models fail in different places. Qwen explores better: it completes more ALFWorld games and more coin collector games, in which the agent has to search for an object or a place, and it repeats itself less. Jev follows instructions more closely: it completes all easy cooking games, which Qwen loses by cooking ingredients that the recipe does not ask to cook, and it chooses the optimal Blackjack action more often. For the use of a System One model inside an agent, these results suggest a division of labor. The model is suited to choosing among options whose consequences are stated, while keeping track of the episode, such as the places already visited or the actions that had no effect, is better left to the surrounding harness than to the history in the request.

6 Limitations

We evaluate one prompt template per dataset without tuning. Other phrasings could change results, in either direction, and some of our weaker results (e.g. AGB-DE, HelpSteer2) may partly reflect our wording. Our reference points are two open-weight models scored with the same prompts, which were written for Jev rather than tuned for these models, and scored in a single forward pass without thinking. Allowing these reasoning LLMs to reason before answering would likely raise their accuracy at a higher cost. Generative proprietary models, which would typically answer in text rather than through option probabilities, are not included, and comparisons with published results remain complicated by differing prompts, splits and in-context examples.

Binary decisions use a fixed 0.5 threshold except in Table 3, where thresholds are tuned on only 1,000 training examples per dataset. Our memorization probes cover only two shallow forms of memorization; they cannot detect memorized question-answer pairs. Several datasets are evaluated on validation rather than test splits because test labels are not public.

We ran each request once and did not measure run-to-run variance. Latency was measured client-side and includes network time. The suite contains no relevance-judgment task, although Jev's Score primitive is a natural fit for graded relevance labels. Our sequential evaluation (Section 5) covers five environments and tests the sensitivity to the wording

of the request only on FrozenLake and BabyAI-Text; agreement with a reference action underestimates step quality where several actions are equally good. Finally, the results refer to `jev-1.13.0`; TypeSafe AI's aliases (`jev-latest`) will likely move to newer versions.

7 Conclusion

Jev delivers what its System One positioning promises on a wide range of short, well-scoped decisions: high zero-shot accuracy on sentiment, topic, intent and commonsense tasks, well-calibrated choice probabilities, confidence scores that support selective prediction, and very low cost. Against two open-weight LLMs scored on identical requests, Jev is ahead of Qwen3.8-27B on most datasets and of Gemma-4-E4B on all, and its calibration matches Qwen's when averaged per dataset.

Used as a step-by-step policy, Jev's strengths carry over only in part. Where the best action follows from the visible state, as in Blackjack, it chooses well and its low-confidence decisions are where its errors lie. Where success depends on keeping track of the episode, it repeats actions that had no effect, revisits dead ends and makes irreversible mistakes, and its action probabilities are overconfident relative to the reference policies. Qwen3.8-27B completes more ALFWorld and TextWorld games on the same requests, but fails in the same ways on BabyAI-Text and FrozenLake, so these failures are not specific to Jev.

Our evaluation of 37 datasets and 346,009 requests also shows its limits. Performance drops sharply for low-resource languages, fine-grained or noisy label sets, legal judgments and rubric-based assessment of generated text. Binary probabilities need application-specific thresholds, especially when many labels are queried at once, and thresholds tuned on a small training sample recover much of the resulting gap. The unexpectedly strong results on MMLU mathematics, which neither open model shows, are not explained by memorized answer positions or option artifacts. We release[4] the suite, the harness and all raw responses so that every number can be recomputed and other typed decision models, as well as generative LLMs, can be compared on identical requests.

Acknowledgments

This research has been partially funded by the Federal Ministry of Education and Research of Germany and the state of North-Rhine Westphalia as part of the Lamarr-Institute for Machine Learning and Artificial Intelligence.

Computations of the responses of Qwen3.8-27B and Gemma-4-E4B were carried out on Marvin[5], an HPC cluster at the University of Bonn.

Anthropic Claude Opus 5.5 [78] was employed to assist in refining and improving the text throughout all sections of this paper. This model was also used to help write the code for this paper, which is available at `github.com/AppliedMachineLearning-Lab/jev-benchmarking`. The authors retain full responsibility for the accuracy, integrity, and originality of the work.

References

[1] Fengrui Liu, Xiao He, Tieying Zhang, Jianjun Chen, Yi Li, Lihua Yi, Haipeng Zhang, Gang Wu, and Rui Shi. Tickit: Leveraging large language models for automated ticket escalation. In *Proc. FSE*, 2025.

[2] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, et al. Llama guard: Llm-based input-output safeguard for human-ai conversations, 2023. URL `https://arxiv.org/abs/2312.06674`.

[3] Liyan Tang, Philippe Laban, and Greg Durrett. MiniCheck: Efficient fact-checking of LLMs on grounding documents. In *Proc. EMNLP*, 2024.

[4] Tobias Deußer, David Leonhard, Lars Hillebrand, Armin Berger, Mohamed Khaled, Sarah Heiden, Tim Dilmaghani, Bernd Kliem, Rüdiger Loitz, Christian Bauckhage, et al. Uncovering inconsistencies and contradictions in financial reports using large language models. In *Proc. BigData*, 2023.

[5] Seungone Kim, Jay Shin, Joel Jang, Shayne Longpre, Hwaran Lee, Sangdoo Yun, Ryan Shin, Sungdong Kim, James Thorne, Minjoon Seo, et al. Prometheus: Inducing fine-grained evaluation capability in language models. In *Proc. ICLR*, volume 2024, 2024.

[4] `github.com/AppliedMachineLearning-Lab/jev-benchmarking`

[5] `hpc.uni-bonn.de/en/systems/marvin`

[6] TypeSafe. Jev: TypeSafe's system one model – documentation. https://docs.typesafe.ai/introduction, 2026. Accessed: 2026-09-29.

[7] Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.

[8] Wenpeng Yin, Jamaal Hay, and Dan Roth. Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach. In *Proc. EMNLP-IJCNLP*, 2019.

[9] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Proc. NeurIPS*, 2020.

[10] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proc. EMNLP Workshop BlackboxNLP*, 2018.

[11] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding, 2021. URL https://arxiv.org/abs/2009.03300.

[12] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R. Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *TMLR*, 2023.

[13] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of language models. *TMLR*, 2023. ISSN 2835-8856.

[14] Stella Biderman, Hailey Schoelkopf, Lintang Sutawika, Leo Gao, Jonathan Tow, Baber Abbasi, Alham Fikri Aji, Pawan Sasanka Ammanamanchi, Sidney Black, Jordan Clive, et al. Lessons from the trenches on reproducible evaluation of language models, 2026. URL https://arxiv.org/abs/2405.14782.

[15] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proc. ICML*, 2017.

[16] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. Language models (mostly) know what they know, 2022. URL https://arxiv.org/abs/2207.05221.

[17] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, et al. Gpt-4 technical report, 2024. URL https://arxiv.org/abs/2303.08774.

[18] Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, Rafael Rafailov, Huaxiu Yao, Chelsea Finn, and Christopher Manning. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback. In *Proc. EMNLP*, 2023.

[19] Mahdi Pakdaman Naeini, Gregory Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using bayesian binning. In *Proc. AAAI*, 2015.

[20] Glenn Brier. Verification of forecasts expressed in terms of probability. *Monthly weather review*, 1950.

[21] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. *Advances in neural information processing systems*, 30, 2017.

[22] Guglielmo Faggioli, Laura Dietz, Charles LA Clarke, Gianluca Demartini, Matthias Hagen, Claudia Hauff, Noriko Kando, Evangelos Kanoulas, Martin Potthast, Benno Stein, et al. Perspectives on large language models for relevance judgment. In *Proc. ICTIR*, 2023.

[23] Paul Thomas, Seth Spielman, Nick Craswell, and Bhaskar Mitra. Large language models can accurately predict searcher preferences. In *Proc. SIGIR*, page 1930–1940, 2024.

[24] Alexander R. Fabbri, Wojciech Kryściński, Bryan McCann, Caiming Xiong, Richard Socher, and Dragomir Radev. Summeval: Re-evaluating summarization evaluation. *TACL*, 2021.

[25] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: NLG evaluation using gpt-4 with better human alignment. In *Proc. EMNLP*, December 2023.

[26] Zhilin Wang, Yi Dong, Olivier Delalleau, Jiaqi Zeng, Gerald Shen, Daniel Egert, Jimmy J Zhang, Makesh N Sreedhar, and Oleksii Kuchaiev. Helpsteer 2: Open-source dataset for training top-performing reward models. *Proc. NeurIPS*, 2024.

[27] Oscar Sainz, Jon Campos, Iker García-Ferrero, Julen Etxaniz, Oier Lopez de Lacalle, and Eneko Agirre. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark. In *Findings of the ACL: EMNLP 2023*, 2023.

[28] Quentin Lhoest, Albert Villanova del Moral, Yacine Jernite, Abhishek Thakur, Patrick von Platen, Suraj Patil, Julien Chaumond, Mariama Drame, Julien Plu, Lewis Tunstall, et al. Datasets: A community library for natural language processing. In *Proc. EMNLP*, 2021.

[29] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. *Proc. NeurIPS*, 2015.

[30] Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. Learning word vectors for sentiment analysis. In *Proc. ACL-HLT*, 2011.

[31] Bo Pang and Lillian Lee. Seeing stars: Exploiting class relationships for sentiment categorization with respect to rating scales. In *Proc. ACL*, 2005.

[32] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proc. EMNLP*, 2013.

[33] Elvis Saravia, Hsien-Chi Toby Liu, Yen-Hao Huang, Junlin Wu, and Yi-Shin Chen. CARER: Contextualized affect representations for emotion recognition. In *Proc. EMNLP*, 2018.

[34] Pekka Malo, Ankur Sinha, Pekka Korhonen, Jyrki Wallenius, and Pyry Takala. Good debt or bad debt: Detecting semantic orientations in economic texts. *J. of the Association for Information Science and Technology*, 2014.

[35] Dogu Araci. Finbert: Financial sentiment analysis with pre-trained language models, 2019. URL `https://arxiv.org/abs/1908.10063`.

[36] Tiago A. Almeida, Jose Maria Gomez Hidalgo, and Akebo Yamakami. Contributions to the study of sms spam filtering: New collection and results. In *Proc. DocEng*, 2011.

[37] Luca Papariello. language-identification. `https://huggingface.co/datasets/papluca/language-identification`, 2021. Hugging Face dataset. Accessed: 2026-09-27.

[38] Dorottya Demszky, Dana Movshovitz-Attias, Jeongwoo Ko, Alan Cowen, Gaurav Nemade, and Sujith Ravi. GoEmotions: A dataset of fine-grained emotions. In *Proc. ACL*, 2020.

[39] Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. Efficient intent detection with dual sentence encoders. In *Proc. Workshop on Natural Language Processing for Conversational AI*, 2020.

[40] Stefan Larson, Anish Mahendran, Joseph J. Peper, Christopher Clarke, Andrew Lee, Parker Hill, Jonathan K. Kummerfeld, Kevin Leach, Michael A. Laurenzano, Lingjia Tang, et al. An evaluation dataset for intent classification and out-of-scope prediction. In *Proc. EMNLP-IJCNLP*, 2019.

[41] David Ifeoluwa Adelani, Hannah Liu, Xiaoyu Shen, Nikita Vassilyev, Jesujoba O. Alabi, Yanke Mao, Haonan Gao, and En-Shiun Annie Lee. SIB-200: A simple, inclusive, and big evaluation dataset for topic classification in 200+ languages and dialects. In *Proc. ACL*, 2024.

[42] Yixin Nie, Adina Williams, Emily Dinan, Mohit Bansal, Jason Weston, and Douwe Kiela. Adversarial NLI: A new benchmark for natural language understanding. In *Proc. ACL*, 2020.

[43] David Ifeoluwa Adelani, Jessica Ojo, Israel Abebe Azime, Jian Yun Zhuang, Jesujoba Oluwadara Alabi, Xuanli He, Millicent Ochieng, Sara Hooker, Andiswa Bukula, En-Shiun Annie Lee, et al. IrokoBench: A new benchmark for African languages in the age of large language models. In *Proc. NAACL-HLT*, 2025.

[44] Yuan Zhang, Jason Baldridge, and Luheng He. PAWS: Paraphrase adversaries from word scrambling. In *Proc. NAACL-HLT*, 2019.

[45] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proc. NAACL-HLT*, 2019.

[46] Lucas Bandarkar, Davis Liang, Benjamin Muller, Mikel Artetxe, Satya Narayan Shukla, Donald Husa, Naman Goyal, Abhinandan Krishnan, Luke Zettlemoyer, and Madian Khabsa. The belebele benchmark: a parallel reading comprehension dataset in 122 language variants. In *Proc. ACL*, 2024.

[47] Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William Cohen, and Xinghua Lu. PubMedQA: A dataset for biomedical research question answering. In *Proc. EMNLP-IJCNLP*, 2019.

[48] Yuzhen Huang, Yuzhuo Bai, Zhihao Zhu, Junlei Zhang, Jinghan Zhang, Tangjun Su, Junteng Liu, Chuancheng Lv, Yikai Zhang, Yao Fu, et al. C-eval: A multi-level multi-discipline chinese evaluation suite for foundation models. *Proc. NeurIPS*, 2023.

[49] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proc. ACL*, 2019.

[50] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 2021.

[51] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge, 2018. URL `https://arxiv.org/abs/1803.05457`.

[52] Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. CommonsenseQA: A question answering challenge targeting commonsense knowledge. In *Proc. NAACL-HLT*, 2019.

[53] Chandra Bhagavatula, Ronan Le Bras, Chaitanya Malaviya, Keisuke Sakaguchi, Ari Holtzman, Hannah Rashkin, Doug Downey, Wen tau Yih, and Yejin Choi. Abductive commonsense reasoning. In *Proc. ICLR*, 2020.

[54] Thomas Hartvigsen, Saadia Gabriel, Hamid Palangi, Maarten Sap, Dipankar Ray, and Ece Kamar. ToxiGen: A large-scale machine-generated dataset for adversarial and implicit hate speech detection. In *Proc. ACL*, 2022.

[55] Todor Markov, Chong Zhang, Sandhini Agarwal, Florentine Eloundou Nekoul, Theodore Lee, Steven Adler, Angela Jiang, and Lilian Weng. A holistic approach to undesired content detection in the real world. In *Proc. AAAI*, 2023.

[56] Zi Lin, Zihan Wang, Yongqi Tong, Yangkun Wang, Yuxin Guo, Yujia Wang, and Jingbo Shang. ToxicChat: Unveiling hidden challenges of toxicity detection in real-world user-AI conversation. In *Findings of the ACL: EMNLP 2023*, 2023.

[57] deepset. prompt-injections. `https://huggingface.co/datasets/deepset/prompt-injections`, 2023. Hugging Face dataset. Accessed: 2026-09-27.

[58] Daniel Braun and Florian Matthes. AGB-DE: A corpus for the automated legal assessment of clauses in German consumer contracts. In *Proc. ACL*, 2024.

[59] Ilias Chalkidis, Abhik Jana, Dirk Hartung, Michael Bommarito, Ion Androutsopoulos, Daniel Martin Katz, and Nikolaos Aletras. Lexglue: A benchmark dataset for legal language understanding in english. In *Proc. ACL*, 2022.

[60] Daniel Cer, Mona Diab, Eneko Agirre, Iñigo Lopez-Gazpio, and Lucia Specia. SemEval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. In *Proc. Workshop SemEval-2017*, 2017.

[61] Bradley Efron and Robert J Tibshirani. *An introduction to the bootstrap*. Chapman and Hall/CRC, 1994.

[62] Qwen Team. Qwen3.8-Max: A new bar for coding and cowork, August 2026. URL `https://qwen.ai/blog?id=qwen3.8`.

[63] Gemma Team, Sherif El Abd, Vaibhav Aggarwal, Robin Algayres, Alek Andreev, Olivier Bachem, Ian Ballantyne, Cormac Brick, Victor Cărbune, Michelle Casbon, et al. Gemma 4 technical report, 2026. URL `https://arxiv.org/abs/2607.02770`.

[64] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proc. EMNLP*, 2020.

[65] Nishant Balepur, Abhilasha Ravichander, and Rachel Rudinger. Artifacts or abduction: How do LLMs answer multiple-choice questions without the question? In *Proc. ACL*, 2024.

[66] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proc. AISTATS*, 2011.

[67] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *Proc. ICLR*, 2021.

[68] Maxime Chevalier-Boisvert, Dzmitry Bahdanau, Salem Lahlou, Lucas Willems, Chitwan Saharia, Thien Huu Nguyen, and Yoshua Bengio. BabyAI: A platform to study the sample efficiency of grounded language learning. In *Proc. ICLR*, 2019.

[69] Thomas Carta, Clément Romac, Thomas Wolf, Sylvain Lamprier, Olivier Sigaud, and Pierre-Yves Oudeyer. Grounding large language models in interactive environments with online reinforcement learning. In *Proc. ICML*, 2023.

[70] Marc-Alexandre Côté, Ákos Kádár, Xingdi Yuan, Ben Kybartas, Tavian Barnes, Emery Fine, James Moore, Matthew Hausknecht, Layla El Asri, Mahmoud Adada, et al. TextWorld: A learning environment for text-based games. In *Computer Games Workshop at IJCAI*, 2018.

[71] Daochen Zha, Kwei-Herng Lai, Yuanpu Cao, Songyi Huang, Ruzhe Wei, Junyu Guo, and Xia Hu. Rlcard: A toolkit for reinforcement learning in card games, 2020. URL https://arxiv.org/abs/1910.04376.

[72] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments, 2024.

[73] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *Proc. ICLR*, 2023.

[74] Xingdi Yuan, Marc-Alexandre Côté, Alessandro Sordoni, Romain Laroche, Remi Tachet des Combes, Matthew Hausknecht, and Adam Trischler. Counting to explore and generalize in text-based games, 2019. URL https://arxiv.org/abs/1806.11525.

[75] Yaodong Yang, Hongyao Tang, Yi Ma, Xingyu Fan, Weixun Wang, Jinpeng Li, and Tianpei Yang. Code owns the simulation, jev owns the evaluation, 2026. URL https://arxiv.org/abs/2610.01834.

[76] Wenqi Zhang, Ke Tang, Hai Wu, Mengna Wang, Yongliang Shen, Guiyang Hou, Zeqi Tan, Peng Li, Yueting Zhuang, and Weiming Lu. Agent-Pro: Learning to evolve via policy-level reflection and optimization. In *Proc. ACL*, 2024.

[77] Davide Paglieri, Bartłomiej Cupiał, Samuel Coward, Ulyana Piterbarg, Maciej Wołczyk, Akbir Khan, Eduardo Pignatelli, Łukasz Kuciński, Lerrel Pinto, Rob Fergus, et al. BALROG: Benchmarking agentic LLM and VLM reasoning on games. In *Proc. ICLR*, 2025.

[78] Anthropic. System card: Claude Opus 5.5. https://www-cdn.anthropic.com/fc1b44717c85dc068bc6ba5024219938094694bd/Claude%20Opus%205.5%20System%20Card.pdf, September 2026. Accessed: 2026-09-26.

A Results per dataset with confidence intervals

Table 10 gives Jev's score on every dataset with its 95% bootstrap interval, the basis for the significance statements in Section 4.2.

Table 10: Main results on the evaluation splits. Score: primary metric (see text); 95% bootstrap confidence interval; ECE of the first Choice or Noul question (– for Score-only tasks).

Dataset	n	Metric	Score↑	95% CI	ECE↓
Text classification					
AG News	7,600	Acc.	0.885	[0.878, 0.891]	0.077
IMDB	25,000	Acc.	0.965	[0.963, 0.967]	0.019
Rotten Tomatoes	1,066	Acc.	0.933	[0.917, 0.947]	0.038
SST-2	872	Acc.	0.964	[0.951, 0.975]	0.015
Emotion	2,000	Acc.	0.585	[0.563, 0.605]	0.279
Financial PhraseBank	970	Acc.	0.730	[0.699, 0.756]	0.138
SMS Spam	5,574	F_1	0.938	[0.924, 0.950]	0.033
Language ID	10,000	Acc.	0.996	[0.995, 0.997]	0.003
GoEmotions	5,427	Macro-F_1	0.243	[0.236, 0.249]	0.176
Intent and topic routing					
Banking77	3,076	Acc.	0.797	[0.782, 0.812]	0.087
CLINC150	5,500	Acc.	0.895	[0.885, 0.902]	0.025
SIB-200	41,820	Acc.	0.815	[0.812, 0.819]	0.045
NLI, paraphrase and grounding					
ANLI	3,200	Acc.	0.739	[0.725, 0.754]	0.102
AfriXNLI	10,800	Acc.	0.640	[0.631, 0.650]	0.165
PAWS	8,000	Acc.	0.850	[0.842, 0.857]	0.040
LLM-AggreFact	29,320	Bal. Acc.	0.786	[0.776, 0.796]	0.128
Reading comprehension and knowledge					
BoolQ	3,270	Acc.	0.913	[0.904, 0.922]	0.043
Belebele	109,800	Acc.	0.867	[0.865, 0.869]	0.015
PubMedQA	1,000	Acc.	0.787	[0.762, 0.812]	0.129
MMLU	14,042	Acc.	0.918	[0.913, 0.923]	0.025
C-Eval	12,342	Acc.	0.839	[0.832, 0.846]	0.014
Commonsense and reasoning					
BIG-bench (MC)	13,227	Acc.	0.814	[0.808, 0.821]	0.026
HellaSwag	10,042	Acc.	0.955	[0.951, 0.959]	0.019
WinoGrande	1,267	Acc.	0.914	[0.899, 0.930]	0.025
ARC (E+C)	3,548	Acc.	0.988	[0.984, 0.991]	0.005
CommonsenseQA	1,221	Acc.	0.882	[0.864, 0.900]	0.031
αNLI (ART)	1,532	Acc.	0.839	[0.819, 0.858]	0.065
Safety, moderation and legal					
ToxiGen	940	Acc. (toxic)	0.878	[0.857, 0.898]	0.046
OpenAI Moderation	1,680	Mean AUPRC	0.717	[0.675, 0.759]	0.036
ToxicChat	5,083	F_1 (toxic)	0.786	[0.753, 0.821]	0.016
Prompt Injections	116	Acc.	0.741	[0.664, 0.828]	0.236
AGB-DE	755	F_1	0.204	[0.122, 0.282]	0.264
UNFAIR-ToS	1,607	Micro-F_1	0.499	[0.449, 0.544]	0.088
Rubric scoring					
STS-B	1,379	Spearman ρ	0.890	[0.879, 0.901]	–
SST-5	2,210	Acc.	0.579	[0.560, 0.599]	–
SummEval	1,600	Mean ρ (summary-level)	0.554	[0.523, 0.571]	–
HelpSteer2	1,038	Mean ρ	0.412	[0.380, 0.444]	–

B Calibration and selective prediction per dataset

Table 11: Choice datasets: accuracy, ECE of the top-option probability, accuracy on the 80% and 50% most confident examples (ranked by Jev's confidence), and area under the risk–coverage curve.

Dataset	Acc.↑	ECE↓	Acc.@80%↑	Acc.@50%↑	AURC↓
Text classification					
AG News	0.885	0.077	0.942	0.953	0.056
IMDB	0.965	0.019	0.993	0.993	0.008
Rotten Tomatoes	0.933	0.038	0.975	0.981	0.024
SST-2	0.964	0.015	0.989	0.995	0.007
Emotion	0.585	0.279	0.642	0.733	0.268
Financial PhraseBank	0.730	0.138	0.796	0.887	0.118
Language ID	0.996	0.003	0.999	1.000	0.001
Intent and topic routing					
Banking77	0.797	0.087	0.888	0.963	0.061
CLINC150	0.895	0.025	0.958	0.979	0.033
SIB-200	0.815	0.045	0.915	0.982	0.046
NLI, paraphrase and grounding					
ANLI	0.739	0.102	0.800	0.869	0.128
AfriXNLI	0.640	0.165	0.686	0.769	0.215
Reading comprehension and knowledge					
Belebele	0.867	0.015	0.955	0.987	0.027
PubMedQA	0.787	0.129	0.868	0.936	0.095
MMLU	0.918	0.025	0.967	0.981	0.026
C-Eval	0.839	0.014	0.927	0.982	0.040
Commonsense and reasoning					
BIG-bench (MC)	0.814	0.026	0.894	0.970	0.054
HellaSwag	0.955	0.019	0.995	0.999	0.004
WinoGrande	0.914	0.025	0.959	0.975	0.034
ARC (E+C)	0.988	0.005	0.998	0.998	0.002
CommonsenseQA	0.882	0.031	0.942	0.980	0.034
αNLI (ART)	0.839	0.065	0.901	0.977	0.051

C TextWorld and BabyAI-Text by setting

Table 12 breaks down the TextWorld and BabyAI-Text results by game type, difficulty and instruction type, next to the number of steps the reference policy needs.

D BabyAI-Text: sensitivity to the request

The prompt of Carta et al. [69] was written for agents trained with reinforcement learning, which learn the effect of each action from the reward. It names the six actions but does not describe them, and it does not state the rules of the environment, for example that moving into an object leaves the agent in place or that a go-to mission is completed by facing the object. To test whether Jev's BabyAI-Text results depend on this, we repeat the 250 missions with the instruction prompt that BALROG [77] uses for zero-shot LLMs on the same level, copied verbatim from its code, added to the state. It states the goal, describes each action in a few words and gives two tips, the second of which reads "It doesn't make sense to repeat the same action over and over if the observation doesn't change." The prompt of Carta et al. [69], the options and the reference are unchanged. Two of BALROG's phrases fit BabyAI only loosely (*pick up* is described as picking up "the object below you", and the first tip suggests *toggle* for objects to pick up); we keep them to test a published prompt rather than one tuned on our episodes.

With the instructions, Jev completes 25.6% of the missions instead of 20.8% (Table 13). On the same missions, 24 are completed only with the instructions and 12 only without them, so the difference is not significant at the 5% level (exact McNemar test, $p = 0.065$). The gain comes from the pick-up missions and those that combine picking up and going to; go-to missions stay at 50%, and open and put-next-to missions stay close to zero. The tip changes Jev's actions: turns make up 28% instead of 7% of its actions, the repeat rate falls from 0.740 to 0.627, and the observation stays the same

Table 12: Jev, Qwen3.8-27B and Gemma-4-E4B on TextWorld and BabyAI-Text by setting. Coin collector levels follow TextWorld's definition: a quest of 10 rooms without dead ends (10), with dead-end rooms off the chain (110), and with dead ends placed randomly (210). Treasure hunter levels 5, 15 and 25 are TextWorld's easy, medium and hard modes. Cooking: easy is one ingredient in one room; medium two ingredients that need cutting and cooking, in six rooms with closed containers and doors; hard three such ingredients in nine rooms. Progress: share of the game's points reached (TextWorld). Steps: mean steps per episode, failed episodes counting the full budget; Ref. steps: the same for the reference policy, which completes every episode. Success is given for all three models; Progress, Step acc. and Steps refer to Jev.

			Success						
Task	Setting	Episodes	Jev	Qwen	Gemma	Progress	Step acc.	Steps	Ref. steps
TextWorld	Coin collector, level 10	10	0.50	0.90	0.20	0.50	0.505	32.7	10.0
	Coin collector, level 110	10	0.10	0.80	0.00	0.10	0.320	49.0	10.0
	Coin collector, level 210	10	0.00	0.10	0.00	0.00	0.290	50.0	10.0
	Treasure hunter, level 5	10	0.80	0.70	0.50	0.80	0.356	13.2	3.0
	Treasure hunter, level 15	10	0.70	0.80	0.10	0.70	0.443	22.8	6.0
	Treasure hunter, level 25	10	0.20	0.80	0.20	0.20	0.236	33.1	11.0
	Cooking, easy	10	1.00	0.20	0.10	1.00	0.711	4.5	3.0
	Cooking, medium	10	0.50	0.30	0.00	0.59	0.510	15.1	11.4
	Cooking, hard	10	0.00	0.00	0.00	0.22	0.229	21.4	20.7
BabyAI-Text	Go to	50	0.50	0.48	0.18	–	0.097	35.3	5.3
	Pick up	50	0.26	0.14	0.06	–	0.076	48.9	6.7
	Open	50	0.08	0.06	0.00	–	0.094	118.9	16.1
	Put next to	50	0.02	0.00	0.00	–	0.107	125.7	11.4
	Pick up, then go to	50	0.18	0.16	0.00	–	0.052	106.5	12.2

Table 13: Jev on the 250 BabyAI-Text missions with the original prompt and with BALROG's instruction prompt added to the state. Forward moves into an obstacle: share of *go forward* actions after the first step that leave the observation unchanged. Top p: mean top-option probability for actions that agree or disagree with the bot.

	Original prompt	+ BALROG instructions
Success	0.208 [0.160, 0.256]	0.256 [0.204, 0.308]
go to / pick up / pick up, then go to	0.50 / 0.26 / 0.18	0.50 / 0.44 / 0.28
open / put next to	0.08 / 0.02	0.02 / 0.04
Step accuracy (all steps)	0.086	0.141
Step accuracy (first step)	0.52	0.50
Repeats	0.740	0.627
Steps with unchanged observation	0.82	0.65
Forward moves into an obstacle	0.96	0.94
Share of turns among actions	0.07	0.28
ECE / Brier	0.469 / 1.132	0.387 / 1.028
Top p correct / incorrect	0.56 / 0.55	0.56 / 0.52

on 65% instead of 82% of the steps. It does not change how Jev moves forward: 94% of its forward moves still run into an obstacle, against 96% without the instructions, and its top-option probability remains almost the same for correct and incorrect actions. The wording of the request therefore accounts for part of the gap, but not for the failure to notice that an action had no effect.

E FrozenLake: sensitivity to the request

To test whether Jev's FrozenLake results reflect the wording of our request, we ask for the best move at a fixed set of 500 positions under four variants of the request (Table 14). The positions are five frozen tiles from each of the first 100 evaluation maps (alternating 4×4 and 8×8), drawn with a fixed seed among the tiles from which the goal can be reached and on which not every move is optimal, each with the full step budget left. Every variant receives the same positions, one request each, so that errors do not compound. *Base* is the request used in the episodes. *Safety* adds to the question that falling into a hole ends the game, that every move can slip sideways, and that a move for which no slip lands on a hole should be preferred, even if it leads away from the goal. *Outcomes* replaces the description of each option ("move up") with the tiles the move can land on and their probabilities, with holes marked. *Deterministic* makes

the ice non-slippery and asks for the move that reaches the goal in the fewest moves without stepping onto a hole; its reference is the first move of a shortest safe path. As a further reference, a scripted policy that plans as if the ice were not slippery reaches 31.2% accuracy on the base positions and 6.9% where the optimal move leads away from the goal.

Table 14: Jev's moves at 500 fixed FrozenLake positions under four variants of the request. Acc.: share of positions where Jev chooses an optimal move; Away: the same on positions where every optimal move leads away from the goal (n=174; n=34 for the deterministic variant). p_{opt}: probability on the optimal moves. Top p: mean top-option probability for correct / incorrect choices. Risky: share of choices that can slip into a hole (deterministic: that step onto a hole). Regret: loss in probability of reaching the goal.

Variant	Acc.↑	Away↑	p_{opt} ↑	Top p	ECE↓	Brier↓	Risky↓	Regret↓
Base	0.320	0.000	0.307	0.71 / 0.72	0.398	1.005	0.394	0.091
Safety	0.370	0.098	0.330	0.53 / 0.52	0.170	0.751	0.368	0.084
Outcomes	0.624	0.615	0.512	0.68 / 0.64	0.084	0.545	0.128	0.020
Deterministic	0.838	0.000	0.808	0.82 / 0.76	0.073	0.238	0.150	–
No-slip planner (scripted, base)	0.312	0.069	0.312	–	–	–	0.400	0.092

Jev reads the map: on non-slippery ice, it chooses the first move of a shortest safe path at 83.8% of the positions. Yet it again chooses only *down* or *right*, at all 500 positions, so it fails at all 34 positions where the shortest safe path starts with a detour up or left, and 15.0% of its moves step directly onto a hole. On slippery ice, stating the strategy changes little: accuracy rises from 32.0% to 37.0% and, where every optimal move leads away from the goal, from 0% to 9.8%. The main effect of this instruction is lower confidence (mean top-option probability 0.53 instead of 0.71), which reduces the ECE from 0.398 to 0.170 without improving the choices. Listing the landing tiles in the options roughly doubles accuracy, to 62.4% overall and 61.5% where the optimal move leads away from the goal, reduces the share of chosen moves that can slip into a hole from 39.4% to 12.8%, and reduces the mean regret from 0.091 to 0.020. This variant does part of the work for the model, since it spells out the consequence of each move one step ahead; it stays below the optimum because the best move also depends on later steps. In all variants, Jev's mean top-option probability is at most 0.06 higher for correct than for incorrect moves.

Together with Blackjack, where Jev chooses the optimal action in 90% of the decisions and its most confident decisions are almost always correct, these results suggest when Jev's step decisions can be trusted. Jev does well when the right action can be recognized from the request, either because it follows from familiar knowledge, as Blackjack strategy does, or because the request states the consequence of each option. It does poorly when the right action has to be derived from rules given in the request and differs from the action that looks most direct, and its probabilities then do not signal the difficulty. The text games add a second weakness: Jev makes little use of its history. On ALFWorld, it repeats the same action with the same result up to 46 times in a row, on TextWorld it re-enters dead ends listed in its history, and on BabyAI-Text it keeps walking into the same obstacle while its request holds three identical observations, also when the request advises against repeating such actions (Section D). Jev appears to choose each action from the current state alone, which would be consistent with training on single decisions. We regard both as hypotheses to be tested on further tasks rather than a general characterization.